Objective

Build an **Ask AI framework** that allows users to interact with a product containing **50 modules, with 10 actions per module — 500 actions in total — through natural language**.

The framework should understand direct requests, indirect intent, complex multi-step tasks, and follow-up questions, then select and execute the appropriate actions across modules.

Ask AI should behave like an agent that completes tasks. **Latency, cost, and quality are equally important design considerations.**

Use Case

A user is working in a CRM or business platform with modules such as Contacts, Conversations, Opportunities, Calendars, Tasks, Campaigns, and Invoices, and asks:

- **Direct:** “Create a contact named Sarah with this email address.”
- **Indirect:** “I keep forgetting to follow up with new leads. Help me stay on top of them.”
- **Complex:** “Find leads created last week who have not replied, exclude anyone with a booked appointment, assign the remaining leads to available sales reps, and create follow-up tasks.”
- **Conditional:** “For unpaid invoices overdue by more than 15 days, draft reminders, but skip customers who already received one this week.”
- **Follow-up:** “Only include leads from the Facebook campaign, and

make those tasks due tomorrow.”

Ask AI should:

- Understand the user’s intent and relevant conversation context.
- Identify the right modules, actions, entities, and parameters.
- Ask a focused clarification when essential information is missing.
- Execute the workflow and clearly report completed, failed, or pending steps.
- Handle unsupported requests honestly instead of inventing capabilities or results.

Requirements

Setup

- **Backend:** Node.js.
- **Frontend:** Vue.js chat interface.
- Use an LLM provider and agent libraries of your choice.
- Define a catalog of **50 realistic business modules and exactly 10 meaningful actions per module**.
- Provide executable handlers for all 500 actions. Stateful mock APIs and generated fixtures are acceptable; actions must return meaningful data or change stored state, rather than always returning success.
- Use synthetic data. Real HighLevel integrations are not required.

- Keep module and action registration extensible so adding capabilities does not require rewriting the core agent.

Core Features

1. Chat-Based Ask AI — Primary Interface

- Support multi-turn conversations and follow-up requests.
- Maintain relevant context, selected entities, filters, and task progress.
- Show useful progress while a workflow runs.
- Present a clear final answer grounded in actual execution results.

2. Module and Action Understanding

- Maintain a searchable catalog of all 500 actions.
- Define each action’s name, description, input schema, output schema, and execution handler.
- Select relevant actions despite overlapping names or similar functionality across modules.
- Resolve entities and validate required parameters before execution.
- Avoid relying on hardcoded mappings from the example prompts.

3. Reasoning and Assistance

Ask AI should:

- Handle explicit commands, indirect requests, typos, and

paraphrases.

- Break complex requests into executable steps across multiple modules.
- Respect dependencies, filters, exclusions, and conditional logic.
- Use prior results to inform subsequent actions.
- Clarify ambiguity without making users repeat information already available.
- Recognize when a request cannot be completed with the available tools.

4. Action Capabilities — Agent Behavior

From chat, Ask AI can:

- Read, search, create, and update business records.
- Combine data from multiple modules.

Example:

“Find new leads who have not booked an appointment and create follow-up tasks.”

→ Search contacts → Check appointments → Filter eligible leads → Create tasks → Summarize the results.

6. Latency, Cost, and Quality — Important

Optimize the framework across all three dimensions and support your choices with measurements.

- **Latency:** Report p50 and p95 time to first useful response and end-to-end completion, separately for direct requests and complex workflows. A loading indicator does not count as a useful response.

- **Cost:** Track input/output tokens, model calls, and estimated model/API cost per request and per successful task, including retries. Document the model, pricing assumptions, cache behavior, and test conditions.
- **Quality:** Measure correct action selection, parameter accuracy, and end-to-end task success. Check actual output and final data state, not just whether the response sounds correct.
- **Evaluation:** Include at least 100 reproducible scenarios spanning all 50 modules, with direct, indirect, multi-step, follow-up, ambiguous, unsupported, and failure cases. Provide expected outcomes and automated assertions. Separately smoke-test all 500 handlers.
- **Trade-offs:** Compare a simple baseline with your optimized approach on the same scenarios. Explain where you improved speed or cost, and any impact on quality.

State your performance targets before running the benchmark. Report failures and missed targets alongside successful results.

Deliverables

1. Code and Implementation

- Working Ask AI chat UI and backend.
- Complete catalog of 50 modules and 500 actions.
- Executable tool handlers, synthetic data, and a reproducible reset/seed process.
- Evaluation suite, execution traces, and benchmark results.

- Steps to install and run locally, including environment variable setup.

2. Demo — 2–5 Minutes

Show:

- A direct request completed through chat.
- An indirect request interpreted correctly.
- A complex workflow involving at least three modules.
- A follow-up that changes the previous request.
- An ambiguity or tool failure handled appropriately.
- Latency, cost, and quality results from the benchmark.

3. Documentation — README

Include:

- Architecture and agent design.
- Module catalog, action schemas, and registration process.
- Tool selection, context management, and execution strategy.
- What is real versus mocked.
- How to run evaluations and reproduce results.
- Latency, cost, and quality trade-offs.
- Known limitations and how the framework would scale beyond 500 actions.

Evaluation Criteria

- **Agent Design:** Understands intent, plans across modules, and completes tasks.
- **Quality and Reliability:** Selects correct actions and parameters; handles ambiguity, failures, and follow-ups.
- **Latency:** Responds and completes workflows quickly, supported by p50/p95 measurements.
- **Cost Efficiency:** Controls model calls, token usage, and cost per successful task.
- **Product and UX Quality:** Provides useful progress, appropriate confirmations, and clear outcomes.
- **Completeness and Code Quality:** Delivers an end-to-end system with all 500 actions, reproducible evaluations, and modular, maintainable code.

Submission

- GitHub repository.
- Demo video.
- README.
- Evaluation dataset and benchmark report covering latency, cost, and quality.